

Case Study: Building a Leaderboard Service

Programming and Data Structures — Week 6, Lecture 4

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to apply this unit's full sorting toolkit to a single, realistic, multi-part system design problem; justify choosing a different sorting algorithm for each distinct sub-problem within that same system, rather than defaulting to one algorithm throughout; identify where the data structures covered in Weeks 2 through 4 — not only the sorting algorithms of Weeks 5 and 6 — are also essential to a complete design; and critique a naive design that simply calls a general-purpose sort everywhere, articulating specifically where and why it falls short.

Outline

Today covers the full problem statement for a game leaderboard service; four distinct sub-problems within it — bulk loading, live insertion, frequent top-N queries, and periodic recomputation; a direct critique of the naive design that uses one general-purpose sort throughout; the fully assembled design, justifying each component's choice individually; an in-class quiz; and a summary.

The problem: a game leaderboard service

Consider designing the backend for a mobile game's leaderboard service, with 1 million players, each holding an integer score between 0 and 10,000. The service has three concrete requirements: loading all 1 million existing scores into ranked order once, at server startup; updating a player's rank whenever their score changes, which happens constantly and live, as players finish matches; and serving "show me the current top 10 players" requests, which happen extremely frequently, likely far more often than scores actually change.

The naive design

The naive design calls `sorted()` on the entire 1-million-player score dictionary every single time a “top 10” request arrives, then takes the first 10 results. Even using Timsort, this costs $O(n \log n)$ per call — for a request pattern that may occur many times per second, re-sorting all 1 million scores from scratch every time is a severe and entirely avoidable cost, since the vast majority of players’ relative rankings do not change between consecutive requests.

Why the range assumption is worth checking, not just assuming

Before committing to counting sort, it is worth explicitly checking rather than assuming that the stated score range genuinely holds. Many real games allow effectively unbounded scores through combo multipliers or special events, in which case k is not actually small relative to n , and Week 5 Lecture 3’s $O(n+k)$ analysis for counting sort breaks down exactly as that lecture warned it would for arbitrary 64-bit integers. The first genuine engineering step in a case like this is verifying the actual data’s range in production, not assuming a simplified problem statement’s stated bound holds without question — a discipline this course has emphasized since Week 2’s insistence on measuring rather than assuming.

Sub-problem 1: initial bulk load

For the initial bulk load of 1 million scores, each an integer in the known, fixed range $[0, 10,000]$, counting sort is the clear winner, exactly per Week 5 Lecture 3’s analysis: $k = 10,000$ is small relative to $n = 1,000,000$, giving $O(n+k)$ time, essentially linear — a one-time $O(n \log n)$ general sort would work correctly but leaves clear performance on the table for no reason, given the range is known in advance.

Sub-problem 2: live score updates

Handling live score updates by re-sorting the entire dataset on every single change repeats exactly the naive design’s central flaw. What is actually needed here is a structure that maintains scores in ranked order continuously, updating efficiently as individual scores change one at a time — this is not a *sorting-algorithm* problem at all, but a *data structure* problem. A balanced binary search tree (Week 7) or a heap (Week 8) maintains an ordering incrementally, at $O(\log n)$ per update, rather than re-deriving the entire order from scratch on every change — this is precisely why this course covers both sorting algorithms and data structures as complementary, not interchangeable, tools.

Sub-problem 3: frequent “top 10” queries

If sub-problem 2’s structure already maintains scores in ranked order continuously, serving a “top 10” query becomes trivial — reading the first 10 elements from an already-ordered structure, costing $O(1)$ or $O(\log n)$ depending on the exact structure chosen, not $O(n \log n)$. The naive design’s core mistake, stated precisely: it treats a *read* query (“show me the top 10, right now”) as though it required redoing all the *write* work (fully re-sorting) on every single call, when in fact the ordering should be maintained incrementally as writes occur, making reads nearly free.

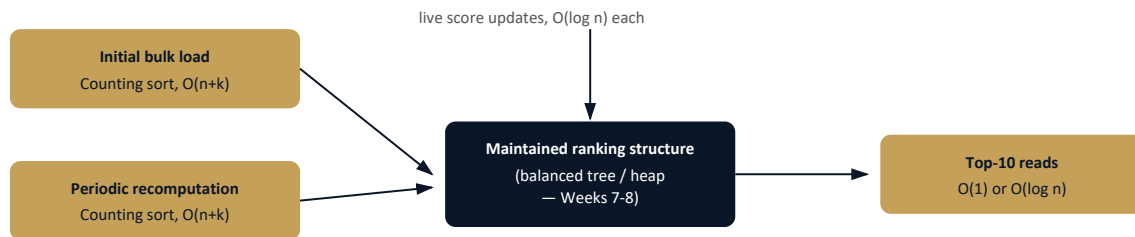
Quantifying the naive design’s actual cost

Putting concrete numbers on the naive design’s cost: with $n = 1,000,000$, $\log_2 n \approx 20$, so a single `sorted()` call costs roughly 20 million operations, using the same recursion-tree-style estimate from Lecture 1. At even a modest 10 “top 10” requests per second, this naive design performs roughly 200 million operations per second purely re-sorting data that, in the vast majority of cases, has not changed since the previous request. The assembled design’s read cost, $O(\log n)$ or $O(1)$ depending on the exact structure, represents a difference of several orders of magnitude — not the kind of marginal improvement a faster sorting algorithm alone could ever close.

Sub-problem 4: periodic full recomputation

Many real systems periodically rebuild their entire ranking from a fresh database snapshot as a safeguard against incremental update logic accumulating small inconsistencies over time. This periodic recomputation is structurally identical to sub-problem 1’s initial bulk load — the same known, fixed score range applies — so the same reasoning and the same choice, counting sort, applies directly, simply run on a recurring schedule rather than only once at server startup.

The picture: four sub-problems, four different tools



Four sub-problems, two categories of tool: sorting algorithms for one-time ordering, a maintained data structure for continuous ordering and cheap reads.

This diagram is the entire case study in one image: a single system with four distinct sub-problems, each solved by the tool this course’s analysis actually justifies for it — not by defaulting to one general-purpose algorithm throughout, which is exactly the naive design’s failure.

Why the naive design is not simply “less optimized”

It is worth being precise that the naive design’s flaw is not merely “using a slightly slower sorting algorithm” — no amount of switching between bubble sort, quicksort, or even a perfectly tuned radix sort fixes sub-problems 2 and 3, because re-sorting from scratch is the wrong *category* of solution for a live, incrementally-updated, frequently-read ranking. The fix is architectural: choosing a structure that maintains order incrementally as data changes, rather than repeatedly re-deriving the entire order from scratch. This is the central, transferable lesson of this entire two-week sorting unit — correctly identifying which *category* of tool a sub-problem calls for comes before optimizing within whichever category has already, correctly or incorrectly, been chosen.

The assembled design, summarized

The fully assembled design uses counting sort for both bulk-load sub-problems, since the known small score range makes it decisively the right sorting algorithm, per Week 5 Lecture 3. For live updates and frequent top-10 reads, the design relies not on a sorting algorithm at all, but on a data structure that maintains order continuously — a preview of exactly what Weeks 7 and 8 build formally. This table is worth comparing directly against the naive design’s single line of code: four sub-problems, two entirely different categories of solution, each individually justified rather than defaulted to.

Connecting back to the whole course's arc, so far

It is worth explicitly tracing this course's arc through this case study. Week 1 provided the language for building structures at all — classes — and the analytical lens for comparing them, Big-O notation. Weeks 2 through 4 covered structures for *storing* data, each with different access trade-offs: arrays' $O(1)$ index access, stacks and queues' disciplined LIFO and FIFO interfaces, and linked lists' $O(1)$ endpoint operations at the cost of index access. Weeks 5 and 6 covered *processes* for putting already-stored data into order. This case study is the first point in the course where all three categories genuinely had to be combined into one coherent design, rather than studied as separate, sequential topics — exactly the kind of synthesis real system design requires.

Exercise

As an exercise, estimate the naive design's total computational cost over one hour of operation, assuming 1,000 “top 10” requests arrive per second, using the $O(n \log n)$ per-call cost established earlier in this lecture, and compare this against the assembled design's cost for the same hour, where reads are $O(\log n)$ or better. Separately, identify one additional realistic requirement this design has not yet addressed — for instance, “show a specific player's current rank out of 1 million,” not merely the top 10 — and sketch, in a few sentences, how you would extend the assembled design to support it.

MTech addition — measuring the real system before redesigning it

Before implementing the assembled design in a real production system, the appropriate engineering step is to profile actual production traffic: what is the real ratio of read requests to write (score-update) requests, and what is the real, measured distribution of score values, not merely the value assumed in this lecture's simplified problem statement. A design justified purely through analysis, without measurement, risks solving a problem the system does not actually have, or solving the right problem with the wrong assumed parameters. This mirrors both Week 2's habit of measuring dynamic array behavior directly and Lecture 3's closing caution about measuring before switching sorting algorithms — applied here at the level of overall system architecture, not merely a single algorithm choice.

Quiz — categorize the fix

Consider a different but structurally similar system: an e-commerce catalog of 500,000 products that is fully re-sorted on every single search request, even though the underlying catalog data only actually changes a handful of times per day. Before checking the answer, determine whether the appropriate fix is “use a faster sorting algorithm” or something categorically different.

Quiz — answer

The appropriate fix is categorically different, not merely a faster sorting algorithm — this is exactly the naive-design pattern from this lecture, restated in a different domain. Since the catalog changes only a few times per day but is queried far more often, the correct fix is to sort once whenever the underlying data actually changes, cache that sorted result, and serve all subsequent read requests directly from the cache until the next real change occurs — no sorting algorithm, however fast, fixes a design that repeats expensive work the underlying data does not justify redoing.

A note on premature architecture, symmetric to Week 4's caution

Just as Week 4 Lecture 4 warned against premature optimization when n is small, a symmetric warning applies here against premature architectural complexity at small scale. A leaderboard serving 50 players does not warrant this lecture's entire assembled design — a single line calling `sorted(scores.items(), key=...)[:10]` genuinely is the correct, simplest answer at that scale, and building the full architecture described here would be solving a problem that does not yet exist. The assembled design earns its added complexity specifically because both conditions established at the start of this lecture actually hold: n is large (1 million) and reads are genuinely frequent — verifying both conditions actually apply, at real production scale, comes before adopting this lecture's more elaborate design.

Summary

Real systems decompose into distinct sub-problems, and this case study demonstrated that each may genuinely call for a different tool — sorting algorithms from Weeks 5 and 6 solve “put this data into order, once”; data structures from Weeks 2 through 4, and Weeks 7 and 8 ahead, solve the different problem of “keep this data in order continuously, as it changes.” The naive-design failure pattern identified here — re-deriving a full ordering on every read, when the underlying data barely changes — recurs across many real systems well beyond this one leaderboard example, and its fix is consistently architectural, never simply a faster sorting algorithm. This closes the two-week sorting unit and, with it, the foundational first half of this course: object-oriented programming, arrays, stacks and queues, linked lists, and now sorting. Next week introduces binary trees, the first structure in this course with a genuinely branching shape, rather than the linear sequence every structure so far has shared.